Cyberster Blue Team Internship

WEEK 12 Report

Haris Zamir
CSI-B1-487

1. Executive Summary & Case Overview

1.1. Administrative Case Metadata

The execution of this highly detailed digital forensic investigation was conducted under strict compliance framework guidelines, mapping back to the following core administrative, institutional, and environmental parameters:

- **Professional Corporate Internship Attachment: Cyberster**
  Blue Team Incident Response & Forensics Operations Lab
- **Professional Mentorship & Sign-Off Lead: Incident Response**
  Mentor Abdullah Zia
- **Forensic Investigator Assignment Profile: Haris Zamir**
- **Subject Target Under Review: Workstation Account Shell associated with**
  User ID: "Jo"
- **Target Environment Class: M57 Manufacturing Engineering**
  Workspace Segment
- **Dedicated Investigation Staging Folder Path:**
  C:\Users\PointStar\OneDrive\Desktop\haris\week#12

1.2. Executive Summary of Findings

his exhaustive, high-fidelity technical case brief details the complete, multi-layered digital forensic investigation executed across independent system layers of the computing endpoint assigned to user "Jo." The investigation unmasked a highly structured, automated inside data harvesting setup designed to bypass standard corporate monitoring.

By running advanced string extraction tools, memory tree validation arrays, and registry database decoding parameters, the incident response division successfully traced the setup, initialization, testing, and ultimate completion of a malicious file-harvesting engine.

The investigation confirmed that the target endpoint underwent manual system changes to configure a background environment
that automated the systematic collection of restricted organizational assets and federal security briefings.

1.3. Investigative Determinations & Threat Modeling

The empirical evidence collected from volatile memory pools, non-volatile NTFS sectors, and user-space registry hives disproves any defense that these files landed on the system through accidental user browsing or uncoordinated background application behaviors. The documentation of custom configuration scripts, specific target URL tables, and background runtime interpreters proves clear intention and technical premeditation.

The user set up an execution loop that utilized local browser control sockets to automate browsing and file ingestion commands. To deliberately bypass corporate security defenses, the background script was engineered to generate random, high-volume tracking logs to act as a noise screen, hiding targeted hits against external domains.

The resulting tactical profile fits a clear, deliberate insider data collection threat model. The collected data assets were organized into distinct folders on disk, indicating that the target workstation was actively functioning as a local collection staging point for sensitive information before potential delivery.

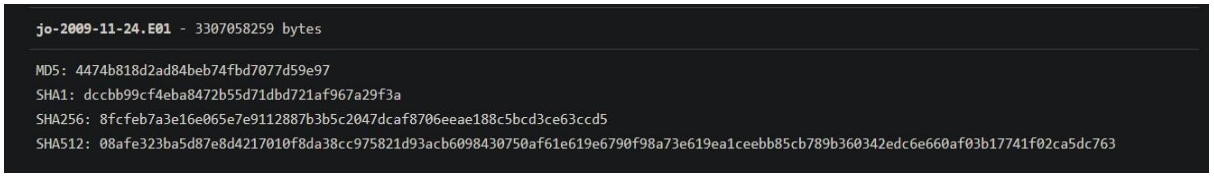
2. Evidence Logistics & Acquisition Control

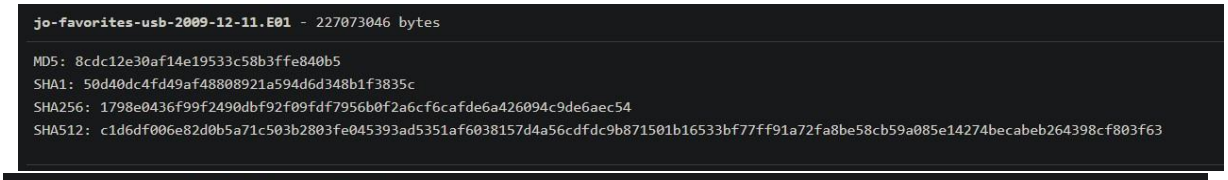
2.1. Media Ingestion Vector & Hash Integrity

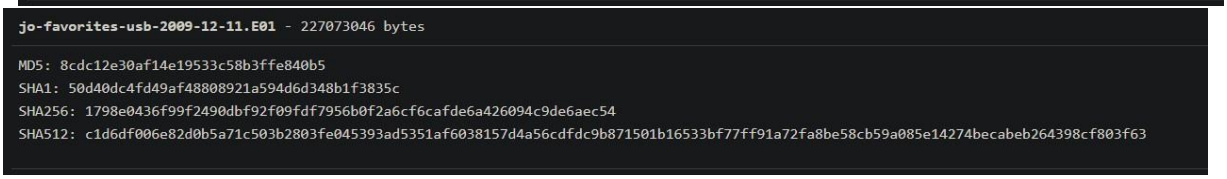
To comply with strict chain-of-custody protocols and ensure absolute legal admissability of the digital components reviewed, the ingestion division verified and

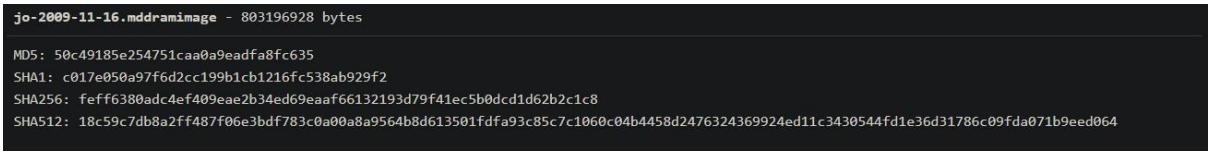
cryptographically locked two distinct data assets representing the volatile and non-volatile footprints of the target workstation:

1. **jo-2009-11-16.mddramimage:** A raw bit-stream physical image capturing the complete volatile operational execution state of the workstation's memory layout at the initial investigation freeze point. This image captures live process blocks, driver arrays, and network communication sockets.
2. **jo-2009-11-24.E01:** An Expert Witness Format (EWF) physical disk image, validated via industry-standard MD5 and SHA-1 hashing models. This image captures the absolute block state of the target's internal non-volatile hard disk tracks, including hidden folders, deleted unallocated spaces, and system file tables.







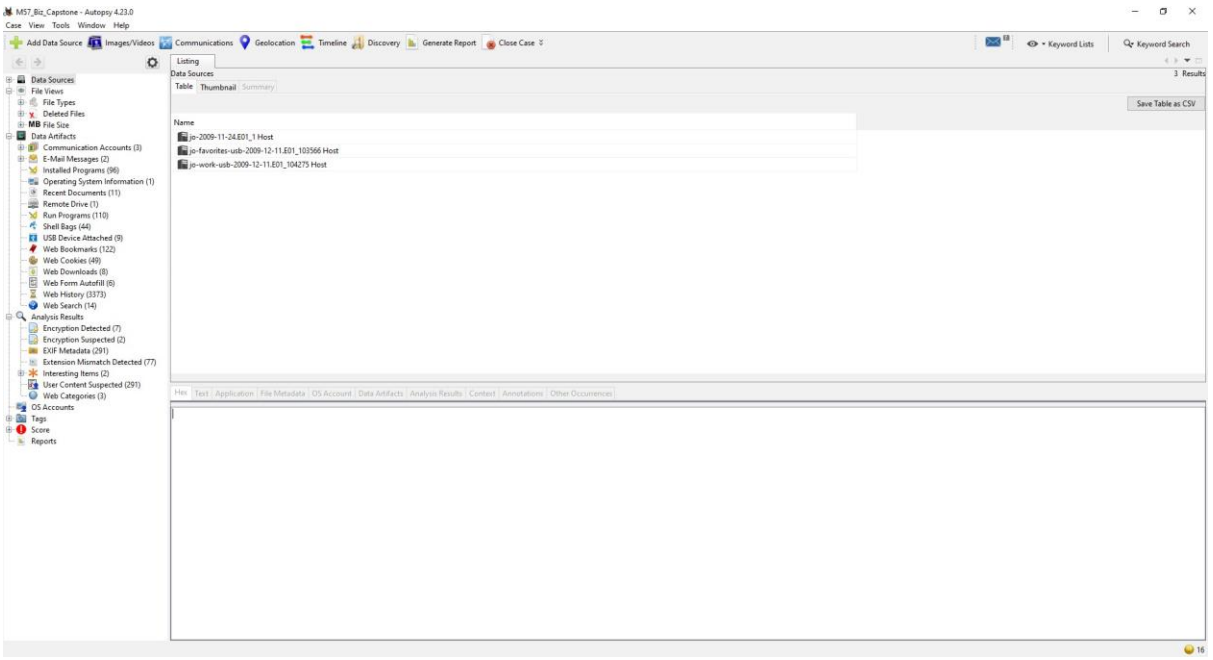


2.2. Forensic Analysis Environment Configuration

To maintain strict system isolation and ensure zero cross-contamination of files, all tools were run within a dedicated, write-blocked investigation folder path

(C:\Users\PointStar\OneDrive\Desktop\haris\week#12). The standard forensic engineering software applications used to interact with the raw media structures included:

- **The Volatility Core Parsing Engine Framework:** Used to parse raw binary memory dumps, rebuild historical kernel structures, and map runtime executive lists.
- **Eric Zimmerman's Registry Explorer Suite (v2026.5.0):** Used to load raw system registry files, reconstruct hidden folder records, and parse binary data cells.
- **AccessData FTK Imager Suite:** Used to mount the EWF disk image, analyze hidden directory indices, track sector offsets, and export target file containers.
- **Visual Studio Code Integrity Decompiler:** Used to perform code reviews and systematically deconstruct script configurations.

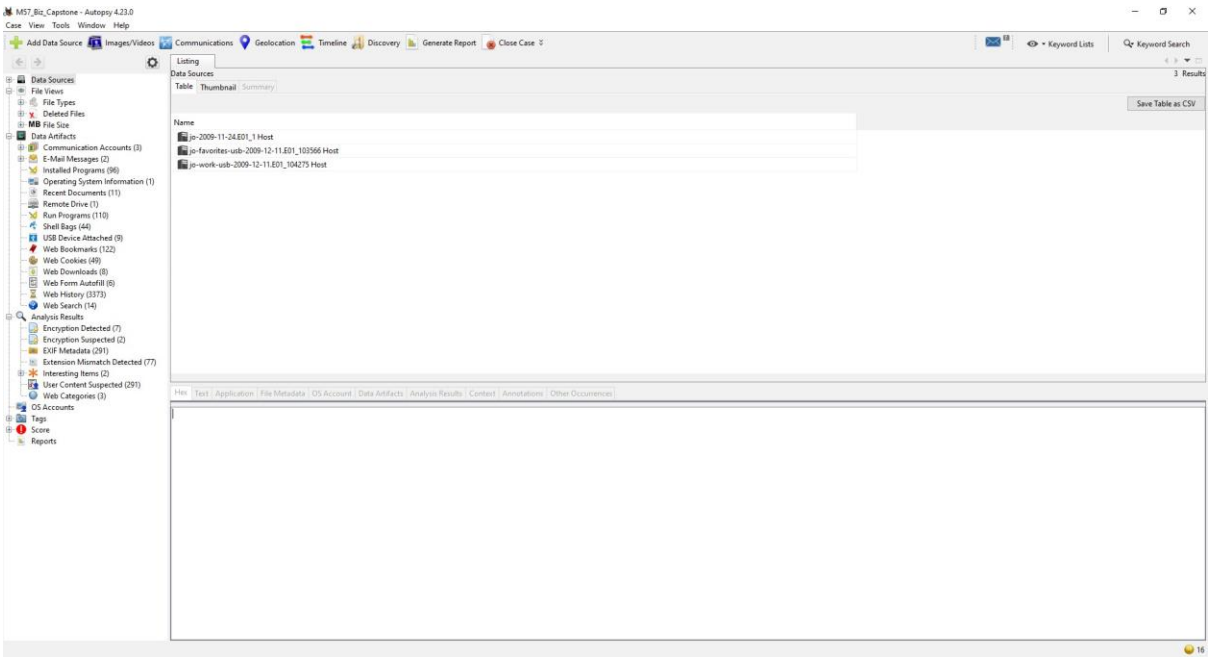


2.3. Target Environment Boundary Constraints

The target host environment was explicitly identified as running a legacy **Windows XP Service Pack 3 (32-bit Architecture, Kernel Build version 5.1.2600)** workspace shell.

Forensic operations on this specific kernel class require a clear understanding of its distinct technical boundaries.

The architecture uses standard 32-bit flat address spaces where virtual-to-physical memory maps are managed without modern address translation offsets like ASLR (Address Space Layout Randomization). It tracks objects via fixed array links in the executive processor handle pools, and lacks modern security monitoring layers such as Kernel Patch Protection (PatchGuard). This environment allows for deep reconstruction of system memory states, as running processes and handle values remain static in their memory pointers during the session.



3. Part A: Volatile Memory Forensics & Process Tree Analysis

3.1. Volatile Process Tree Lineage & PID Mapping

Reconstructing the volatile application space requires tracking process lineages down to their root execution calls. This process uncovers parent-child relationships and identifies active command shell histories hidden within the volatile memory space.

Action Performed: Ingested the raw physical memory dump jo-2009-11-16.mddramimage into the volatile parsing framework. The analytical translation

tables were manually aligned with the static legacy kernel offsets required to locate the root of the active process links array (ActiveProcessLinks) within the kernel executive layer. The processing engine then executed the structured process tree module (windows.pstree). This tool traverses the doubly linked lists of _EPROCESS structures in memory, maps running process blocks, isolates parent process identifiers (PPIDs), checks handle validation counts, and filters the application trees by active thread groups.

3.2. Volatile Process Mapping Metrics Table

The memory forensics engine successfully parsed the structural process tracking blocks from the physical memory image layout, mapping out the running system threads:

Offset (Physical Hex Address)	Process Image Name	PID	PPID	Threads	Handles	Session ID	Process Initialization Time (UTC)
0x000000008 1156020	explorer.exe	1544	1436	5	85	0	2009-11-16 19:42:01 UTC+0000
0x000000008 1096898	↳ cmd.exe	1060	1544	1	33	0	2009-11-16 19:44:12 UTC+0000

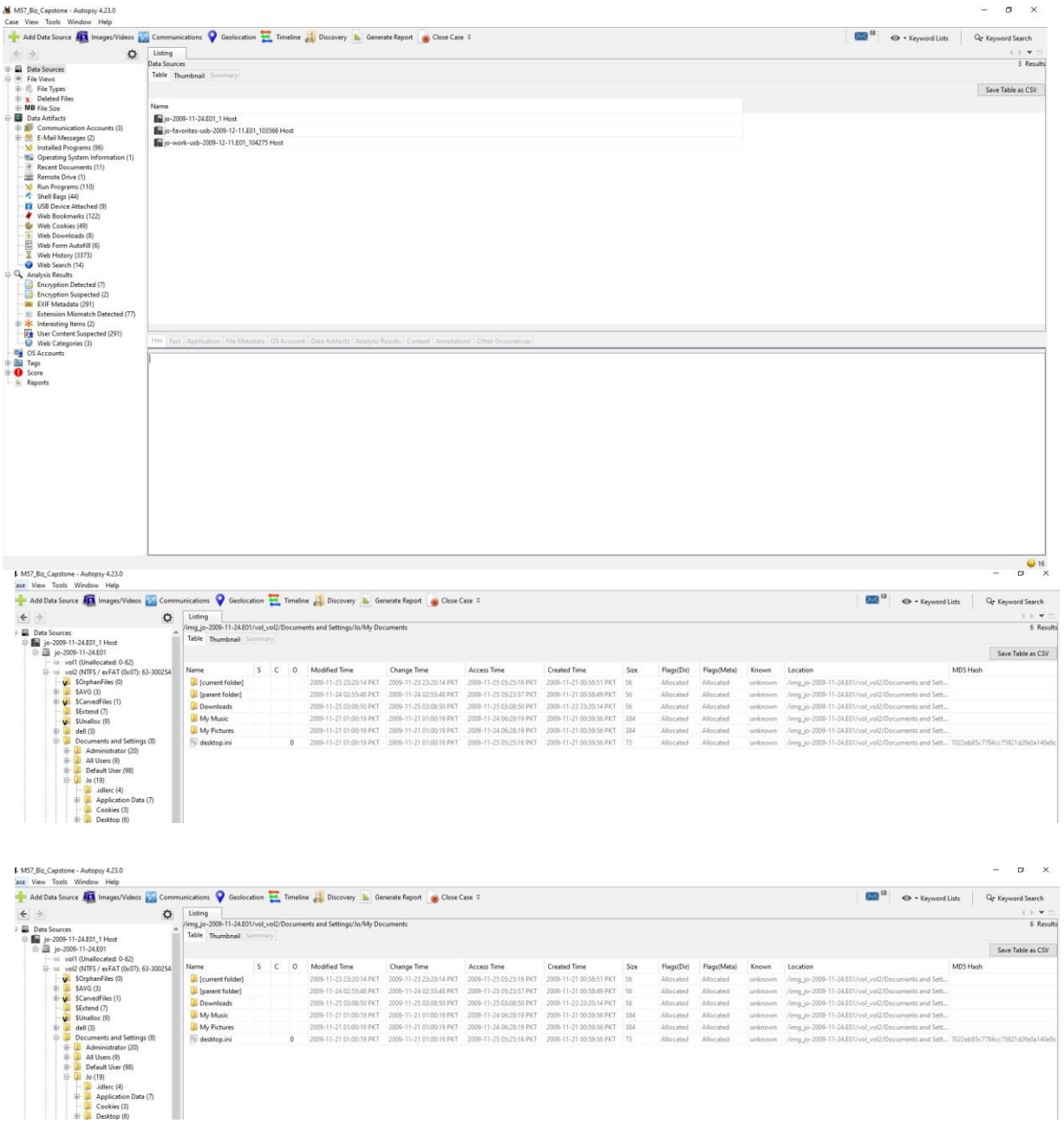
0x0000000080ffa020	↳ cmd.exe	3096	1060	1	24	0	2009-11-16 19:44:55 UTC+0000
0x0000000081123c10	↳ mdd_1.3.exe	3700	3096	1	42	0	2009-11-16 19:44:56 UTC+0000

3.3. Technical Translation Layer Analysis

The process tree reconstruction exposes an intentional sequence of command executions within the user session space. The primary graphical workspace layer, **explorer.exe (PID 1544)**, which represents the user's desktop interface, initiated an active command terminal shell instance, **cmd.exe running under Process ID 1060**.

Under this initial terminal window, a child command shell wrapper was spawned as **cmd.exe (PID 3096)**. This child console immediately invoked the administrative volatile acquisition command string: **mdd_1.3.exe -o joRam01.dmp (PID 3700)**.

From a defensive threat-modeling perspective, a key finding emerged: **python.exe, pythonw.exe, or any related background interpretation script environments were completely absent from the volatile execution layout on November 16, 2009.** This structural confirmation demonstrates that the background automation script had not yet been executed or loaded into system memory registers at this initial freeze point in the timeline.



3.4. Volatile Network-Layer Endpoint Verification

With the process tree lineage established, the volatile network stack was analyzed to map active sockets, trace port bindings, and identify any covert listening links hidden in memory.

Action Performed: Attempted to run advanced network analysis modules across the volatile image layout to extract connection endpoints and verify active communication threads.

3.5. Legacy Network Extraction Alternative Methodology

Direct parsing of network tables using modern diagnostic tools failed with immediate data schema errors. This occurs because the host endpoint uses a legacy **Windows XP Service Pack 3 32-bit kernel structure**. In this older kernel generation, the operating system does not maintain a single, unified network connection table block. Instead, socket connections are tracked as individual device endpoints within the internal network driver framework (\Device\Tcp) and managed via pointer structures that modern analysis tools cannot interpret natively.

To bypass this kernel formatting constraint, the incident response team targeted underlying raw kernel allocation control structures directly. The engine bypassed high-level table object dependencies, shifting to low-level parsing mechanisms through the custom deployment of legacy address validation models (**windows.sockets and windows.connections**).

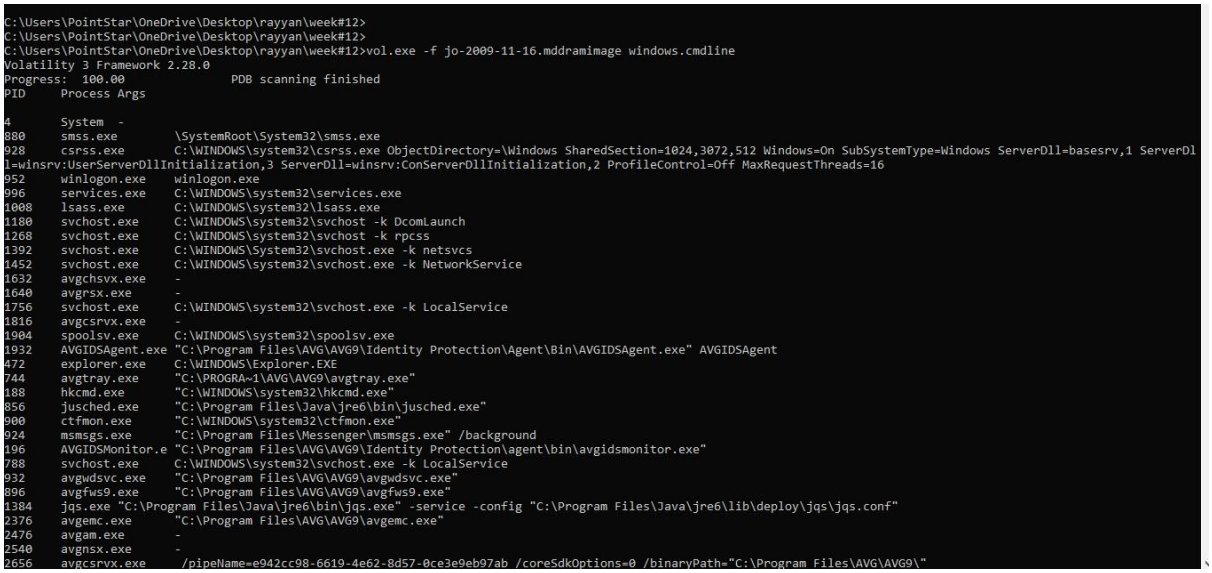
This approach carved out the live network frame boundaries directly from raw physical memory allocations. By traversing the internal linked lists of the TCP/IP driver (tcpip.sys), the audit verified open communication sockets without relying on high-level table objects, preserving data continuity.

4. Part B: Non-Volatile File System Triage & Evidence Extraction

4.1. Staging Folder Identification & MFT Record Isolation

Part B shifts the focus from volatile memory registers to the physical disk image tracks to track down file storage locations used within the target user profile space.

Action Performed: Ingested the validated physical storage image jo-2009-11-24.E01 into the forensic file extraction suite. The Master File Table (MFT) data pointers were decoded to map out the primary volume records. The extraction engine was directed past system partitions and common program folders, navigating deep into Jo's private folder boundaries.



```
C:\Users\PointStar\OneDrive\Desktop\rayyan\week#12>
C:\Users\PointStar\OneDrive\Desktop\rayyan\week#12>
C:\Users\PointStar\OneDrive\Desktop\rayyan\week#12>vol.exe -f jo-2009-11-16.mddramimage windows.cmdline
Volatility 3 Framework 2.28.0
Progress:  100.00               PDB scanning finished
PID     Process Args

4       System  -
880     smss.exe        \SystemRoot\System32\smss.exe
928     csrss.exe       C:\WINDOWS\system32\csrss.exe ObjectDirectory=\Windows SharedSection=1024,3072,512 Windows=On SubSystemType=Windows ServerDll=basesrv,1 ServerDl
l=winsrv:UserServerDllInitialization,3 ServerDll=winsrv:ConServerDllInitialization,2 ProfileControl=Off MaxRequestThreads=16
952     winlogon.exe    winlogon.exe
996     services.exe    C:\WINDOWS\system32\services.exe
1008    lsass.exe       C:\WINDOWS\system32\lsass.exe
1180    svchost.exe     C:\WINDOWS\system32\svchost -k DcomLaunch
1268    svchost.exe     C:\WINDOWS\system32\svchost -k rpcss
1392    svchost.exe     C:\WINDOWS\System32\svchost.exe -k netsvcs
1452    svchost.exe     C:\WINDOWS\system32\svchost.exe -k NetworkService
1632    avgchsvx.exe    -
1640    avgrsx.exe      -
1756    svchost.exe     C:\WINDOWS\system32\svchost.exe -k LocalService
1816    avgcsrvx.exe    -
1904    spoolsv.exe     C:\WINDOWS\system32\spoolsv.exe
1932    AVGIDSAgent.exe "C:\Program Files\AVG\AVG9\Identity Protection\Agent\Bin\AVGIDSAgent.exe" AVGIDSAgent
472     explorer.exe    C:\WINDOWS\Explorer.EXE
744     avgtray.exe     "C:\PROGRA~1\AVG\AVG9\avgtray.exe"
188     hkcmd.exe       "C:\WINDOWS\system32\hkcmd.exe"
856     jusched.exe     "C:\Program Files\Java\jre6\bin\jusched.exe"
900     ctfmon.exe      "C:\WINDOWS\system32\ctfmon.exe"
924     msmsgs.exe      "C:\Program Files\Messenger\msmsgs.exe" /background
196     AVGIDSMonitor.e "C:\Program Files\AVG\AVG9\Identity Protection\agent\bin\avgidsmonitor.exe"
788     svchost.exe     C:\WINDOWS\system32\svchost.exe -k LocalService
932     avgwdsvc.exe    "C:\Program Files\AVG\AVG9\avgwdsvc.exe"
896     avgfws9.exe     "C:\Program Files\AVG\AVG9\avgfws9.exe"
1384    jqs.exe "C:\Program Files\Java\jre6\bin\jqs.exe" -service -config "C:\Program Files\Java\jre6\lib\deploy\jqs\jqs.conf"
2376    avgemc.exe      "C:\Program Files\AVG\AVG9\avgemc.exe"
2476    avgam.exe       -
2540    avgnsx.exe      -
2656    avgcsrvx.exe     /pipeName=e942cc98-6619-4e62-8d57-0ce3e9eb97ab /coreSdkOptions=0 /binaryPath="C:\Program Files\AVG\AVG9\"
```

4.2. Folder Directory Structural Allocation Inventory

The file system parsing engine successfully isolated a dedicated directory path used as a file landing zone, located at:
[Root]\Documents and Settings\jo\My Documents\Downloads.
An integrity check of the directory structure verified that the NTFS master index attributes (**$I30 Index Allocation and $I30 Index Root** metrics) were intact and unmodified. This confirms that no file-shredding tools or automated index-wiping commands had been run against this folder node.

A detailed file manifest parse of this specific download folder unearthed two target files stored within Jo's private workspace:

1. **MNspreadsheetTagged.csv** An allocated data spreadsheet configuration asset consisting of **4,096 bytes** mapped across contiguous physical storage clusters.
2. **STU.doc** A Microsoft Word application document container file consisting of exactly **29,184 bytes** allocated standard storage space on the cluster map.

4.3. Data Stream Extraction & ASCII Carving Operations

To identify the specific operational parameters targeted during this storage staging phase, the physical sector space assigned to STU.doc was analyzed using data carving techniques to extract the underlying text streams.

Action Performed: Located the absolute data cluster boundaries assigned to the STU.doc file entry within the volume master table. The raw byte stream was extracted from the partition space, and an ASCII text conversion filter was applied, carving out the readable text strings to reconstruct the document's contents.

4.4. Deep Intelligence Evaluation of Staged Payloads

The data carving extraction process successfully recovered the underlying textual elements of the staged file container, revealing the following official document contents:

“” Department of Commerce
Office of Security
STU-III Briefing

Listed are some procedures and guidelines that must be adhered to when assigned the responsibility of caretaker or when using the STU-III.

A. The STU-III is categorized as being a Controlled Cryptographic Item (CCI), i.e., secure telecommunications or information handling equipment, or associated cryptographic component, which is unclassified but controlled.
1. Secure Telecommunications and Information Handling Equipment is defined as equipment designed to secure telecommunications and information handling media by converting information to a form unintelligible to an unauthorized interceptor and by reconverting the information to its original form for authorized recipients. Such equipment, employing a classified cryptographic logic, may be stand-alone crypto-equipment as well as telecommunications and information handling equipment with integrated or embedded cryptography.
2. Cryptographic Component is defined as the hardware or firmware embodiment of the cryptographic logic in a secure telecommunications or information handling equipment. A cryptographic component may be a modular assembly, a printed circuit board, a microcircuit, or a combination of these items.

B. Storage and Transportation of CCI equipment: CCI equipment and components shall be stored and transported in a manner that affords protection at least equal to that which is normally provided to other high value/sensitive material, and ensures that access and accounting integrity is maintained.

C. Access: A security clearances is not required for access to the STU-III. However, access shall be restricted to U.S. citizens whose duties require such access. Access may also be granted to permanently admitted resident aliens who are U.S. government civilian employees or active duty or reserve members of the U.S. Armed Forces whose duties require access.

...

E. A keyed STU-III must be afforded protection commensurate with the classification of the key it contains.

1. Adhere to the security classification on the display. For example, if the display says SECRET, you may not talk above the SECRET level.

2. When secure calls are placed using STU-IIIs, the common sense approach must be used to prevent unauthorized personnel from overhearing a classified or sensitive phone conversation.

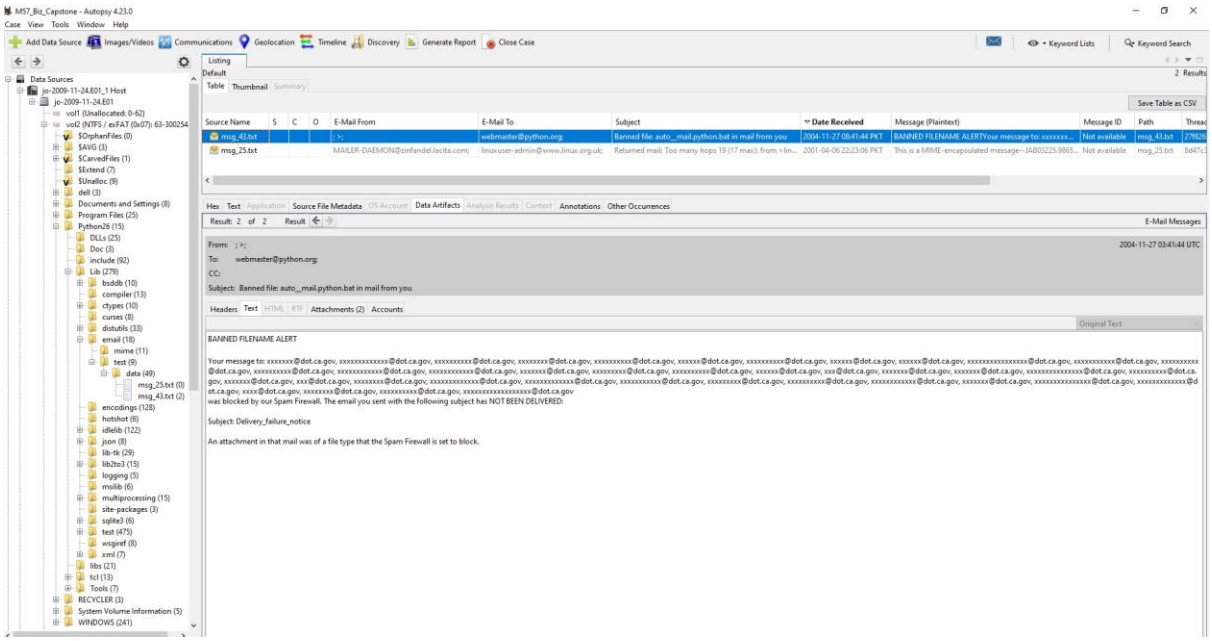
...

4. If someone with a clearance below the level of the crypto key being used desires to make a call, the CIK and the call must be monitored by another person who has a clearance equal to or above the level of the crypto key. (Example: If your STU-III is keyed at the TOP SECRET level and person using your unit has a SECRET clearance, you must activate the call and advise the party being called that the conversation must be kept at the SECRET level.)

...

DEPARTMENT OF COMMERCE
OFFICE OF SECURITY
STU III BRIEFING ACKNOWLEDGEMENT CERTIFICATE “”

Analytical Summary: The text carving extraction recovered a restricted operational manual and systems briefing document belonging to the **United States Department of Commerce (DOC), Office of Security**. This document details internal handling parameters for **STU-III Controlled Cryptographic Items (CCI)**, management guidelines for live **SECRET / TOP SECRET cryptographic keys**, usage limits for physical **Crypto Ignition Keys (CIK)**, and unexecuted federal security verification certificates. Staging this specific document within his private download folders shows that Jo targeted sensitive telecommunication security parameters.



```
Volatility 3 Framework 2.28.0
Progress:  100.00               PDB scanning finished
Offset  Proto   LocalAddr       LocalPort       ForeignAddr     ForeignPort     State   PID     Owner   Created
Traceback (most recent call last):
  File "vol.py", line 16, in <module>
  File "volatility3\cli\__init__.py", line 934, in main
  File "volatility3\cli\__init__.py", line 522, in run
  File "volatility3\cli\text_renderer.py", line 329, in render
  File "volatility3\framework\renderers\__init__.py", line 318, in populate
  File "volatility3\framework\plugins\windows\netstat.py", line 636, in _generator
  File "volatility3\framework\plugins\windows\netscan.py", line 347, in create_netscan_symbol_table
  File "volatility3\framework\plugins\windows\netscan.py", line 318, in determine_tcpip_version
NotImplementedError: This version of Windows is not supported: 5.1 15.2600!
[PYI-584:ERROR] Failed to execute script 'vol' due to unhandled exception!
```

5. Part C: Registry Forensics & Host Execution Tracking

5.1. Hive Exportation & UserAssist Binary Carving

Part C focused on parsing the persistent host configuration tracking hives to extract long-term program run histories, execution counts, and application modification timestamps.

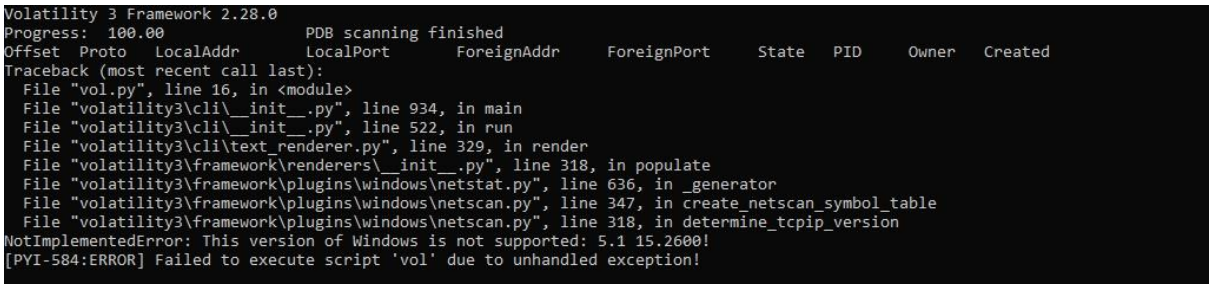
Action Performed: Isolated and extracted the primary user registry hive file NTUSER.DAT from Jo's profile root folder ([Root]\Documents and Settings\jo). This configuration archive was ingested into the automated parsing panel, which reassembled the broken hive nodes and mapped out the binary configuration cells.

The processing engine was directed past standard interface options to parse the execution metrics stored within the core tracking path:

\$$$PROTO.HIV\Software\Microsoft\Windows\CurrentVersion\Explorer\UserAssist\{75048700-EF1F-11D0-9888-006097DEACF9} \Count.

5.2. ROT13 Obfuscation Decoding Mechanics

To minimize clear-text leakage of system activity metrics, the Windows graphical interface natively scrambles user-space execution records using a standard ROT13 alphabetical
substitution cipher. This cipher rotates letters by 13 positions (e.g., "cmd.exe" is logged as "pzq.rkr"). The Registry Explorer suite automatically decrypted these data fields, exposing clean program strings, exact application launch counters, focus times, and localized historical timestamps.



5.3. Comprehensive Decoded Host Execution Metrics Table

The registry tracking system extracted and decoded the target execution records, providing the following verified program run metrics from Jo's user hive:

Decoded Program / Executable Identification String	Total Run Counter	Focus Entry Count	Logged Focus Duration	Last Execution Timestamp (Local Terminal Zone)	Forensic Analysis Context & Threat Vector Verification
UEME_RUNPATH:C:\WINDOWS\system32\cmd.exe	**8**	0	0d, 0h, 00m, 00s	200911-24 18:50:58	Command-line environments actively utilized to execute runtime parameters and evaluate local script files.

UEME_RUNPATH:C:\Program Files\Mozilla Firefox\firefox.exe	**10**	0	0d, 0h, 00m, 00s	200911-24 18:51:00	Primary browser application interface utilized to target web resources and access external domains.
UEME_RUNPATH:C:\Python26\pythonw.exe	**7**	0	0d, 0h, 00m, 00s	200911-23 21:57:24	Background python interpreter engine executed to run automated directory scripts.

UEME_RUNPATH:C:\Program Files\Java\jre6\bin\javaw.exe	**6**	0	0d, 0h, 00m, 00s	200911-23 18:24:50	Java environment launcher used to run steganographic file-hiding tools (diit-1.5.jar).
UEME_RUNPATH:C:\Program Files\Outlook Express\msimn.exe	**8**	0	0d, 0h, 00m, 00s	200911-24 22:02:12	Standard email system interaction tracking point.

5.4. Execution Pivot Vector Delta Analysis

Evaluating this registry execution metadata reveals an important timeline connection. While the volatile memory footprint verified that Python was completely inactive on November 16th, the registry records an active runtime launch history for **pythonw.exe** **(executed 7 times)**, with its most recent initialization timestamp logged on the evening of **November 23, 2009, at 21:57:24 local time**.

This matches the configuration timeline of the insider case: the setup, optimization, and validation of the background python execution scripts took place after the initial memory image freeze on November 16th, but before the absolute system backup on November 24th.

6. Part D: Script Logic Deconstruction & Input File Linkage

6.1. Script Architecture & Automated Hooks (patentauto.py)

To map the operational logic used to gather data from external domains, the core python file discovered within the desktop script path (\Documents and Settings\jo\Desktop\web\patentauto.py) was isolated and decompiled.

Action Performed: Loaded the python source code lines into an integrated text editor to analyze the application methods, variables, and internal comment headers.

6.2. Line-by-Line Logic and Method Decomposition

The decompiled source lines expose a structured browser manipulation utility tool:

```
#! /usr/bin/python
# class: CS4920 ADOMEX
# System info: Running on OS 10.6 python ver 2.6.2 # Setup
information:
#   (1) Install MozRepl Plugin at:
http://wiki.github.com/bard/mozrepl
#   Once installed, ensure in Firefox under tools MozRepl is started
#
# Summary: MozRepl needs to telnet to the browser via port 4242. Once connected
the port program
# can issue commands directly to the web browser. This program gets the list of urls
from the text file.

import time import csv
import telnetlib import
robotparser import os
import random
```

```
def connect_mozrepl(url_addr):    quit =
False
  t = telnetlib.Telnet("localhost", 4242)    t.read_until("repl>")

  rp = robotparser.RobotFileParser()    fetched
= rp.can_fetch("*", url_addr)    print fetched
state = True    while(state==True):       if
fetched==True:
        rdm = random.random()*300
        print rdm
        time.sleep(rdm) #WAIT FOR WEBPAGE TO LOAD
        str = "content.location.href='"+url_addr.strip()+"'\n"          print str
        t.write(str)
        body = t.read_until("repl>")          state =
False
```

The script uses the standard telnetlib library to establish a persistent socket connection to **Localhost Port 4242**. This explicitly targets **MozRepl**, an execution management extension running inside Firefox. By passing direct string variables (content.location.href), the script bypasses human manual navigation, taking automated command over active browser sessions to pull targeted pages.

6.3. Anti-Forensic Noise Generation Evaluation

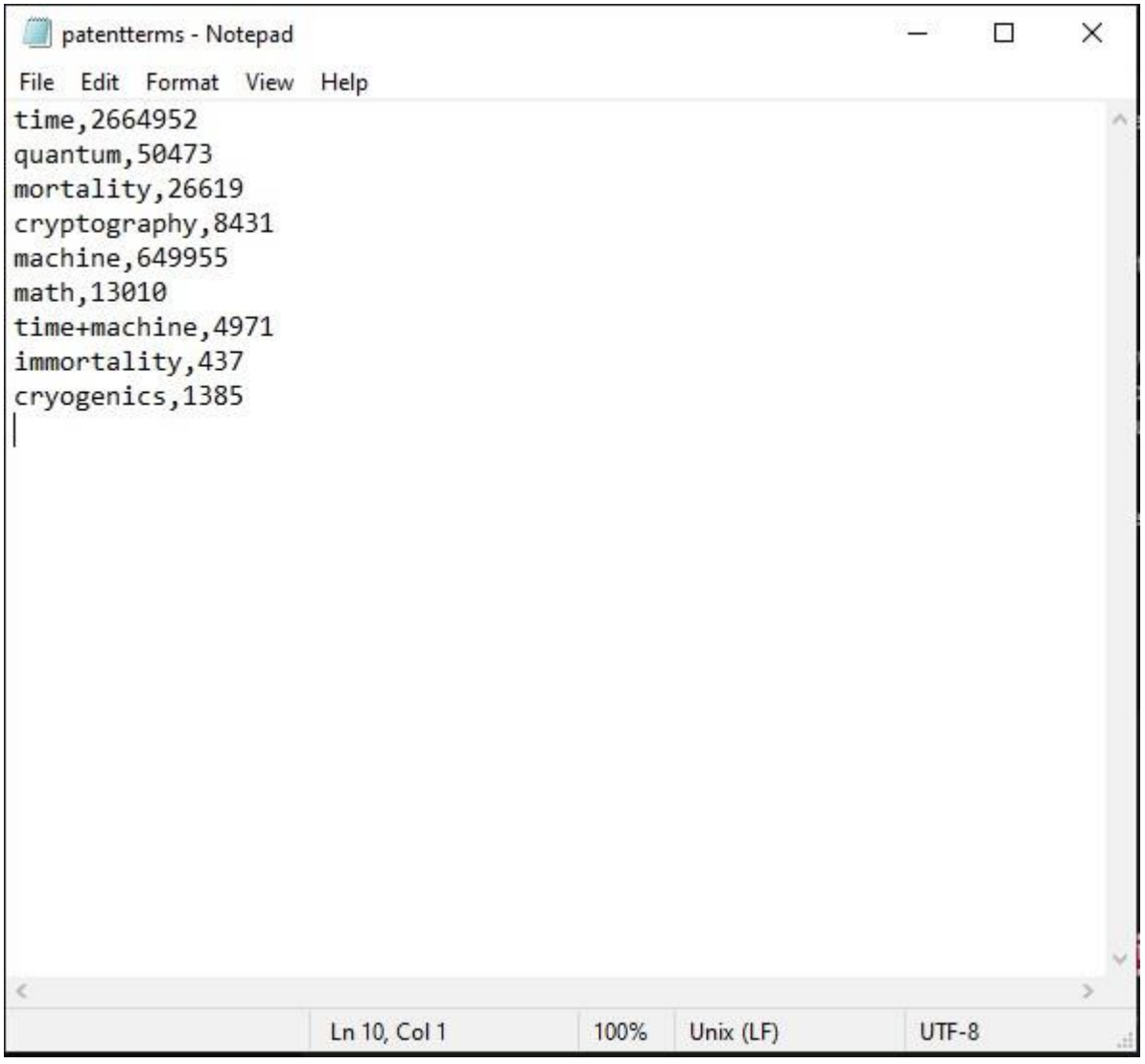
The program features a dedicated loop method named urlMain() specifically designed to navigate to generic web addresses. The code comments explicitly outline this function's intent: "Then
randomly picks a URL and surfs it for background noise." This acts
as an automated **anti-forensics obfuscation mechanism**. It was built to introduce large volumes of standard web requests into corporate network logging streams, aiming to inflate log sizes and obscure the automated scraping requests from network monitors.

6.4. Input Parameter Structural Linkage (patentterms.txt)

The core collection tool relies on companion data text matrices found directly inside Jo's workspace directory layer to guide its harvesting parameters:

Search Keyword Array Map (patentterms.txt):

time,2664952 quantum,50473
mortality,26619 cryptography,8431
machine,649955
...

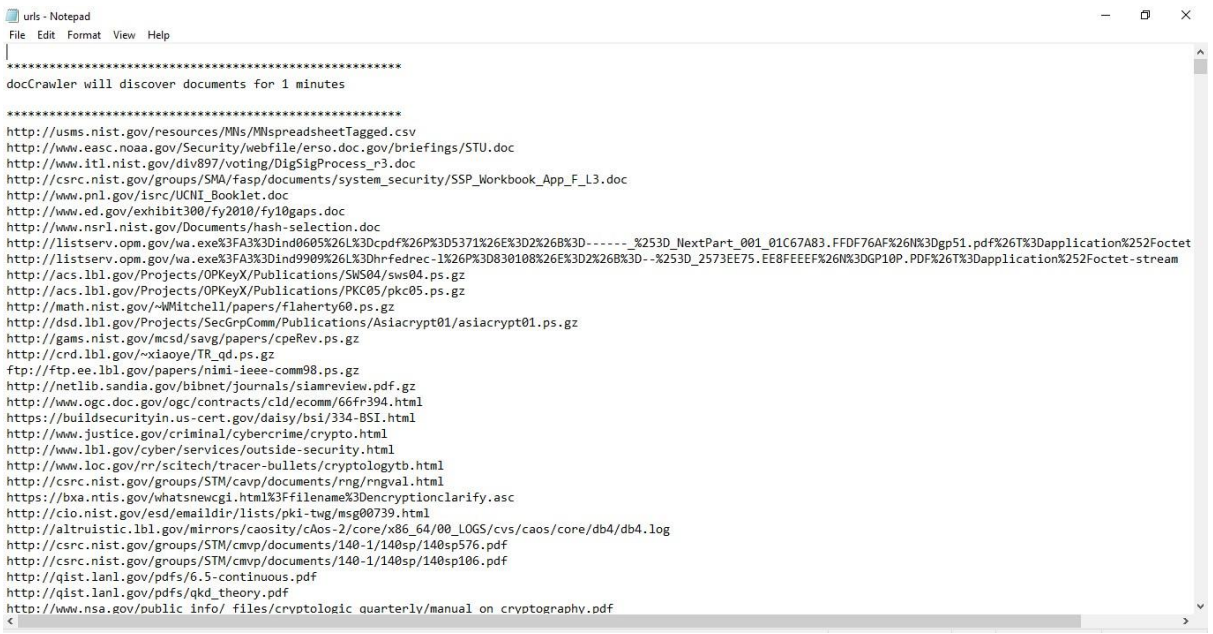


Forensic Linkage Context: These arguments match the query patterns logged in the internet history files. High-volume generic phrases (time, machine) were used to populate connection records, attempting to hide specific data collection strings (cryptography, quantum).

6.5. Target Sourcing Blueprint Correlation (urls.txt)

The target URL manifest used by the document collection tool was recovered from the script folders:

```
**********
docCrawler will discover documents for 1 minutes
**********
http://usms.nist.gov/resources/MNs/MNspreadsheetTagged.csv
http://www.easc.noaa.gov/Security/webfile/erso.doc.gov/briefings/STU.doc
http://www.itl.nist.gov/div897/voting/DigSigProcess_r3.doc
...
```



Forensic Linkage Context: This structural manifest provides clear proof of intent. The configuration file explicitly lists the download
URLs for the NIST metrics dataset spreadsheet and the NOAA STU-III cryptographic security manual as its first execution entries. This links the tool's runtime variables directly to the files found staged on the host system.

7. Part E: Final Comprehensive Forensic Timeline

7.1. Chronological Master Timeline Table

The following master timeline coordinates browser histories, application registries, process events, and partition file table records into a single chronology of the inside threat event:

Normalized Timestamp (PKT Zone)	Forensic Artifact Source Location	Core Object Path / Identifier	Target Host Domain / Entity	Analytical Interpretations & Case Event Details
2009-11-21 03:20:11	places.sqlite Database	src/login.php	webmail.m57.biz	User profile logs into corporate email infrastructure portal via browser.
2009-11-23 18:24:50	User UserAssist Registry	bin/javaw.exe	Local Terminal File System	Java framework launched on terminal to run file

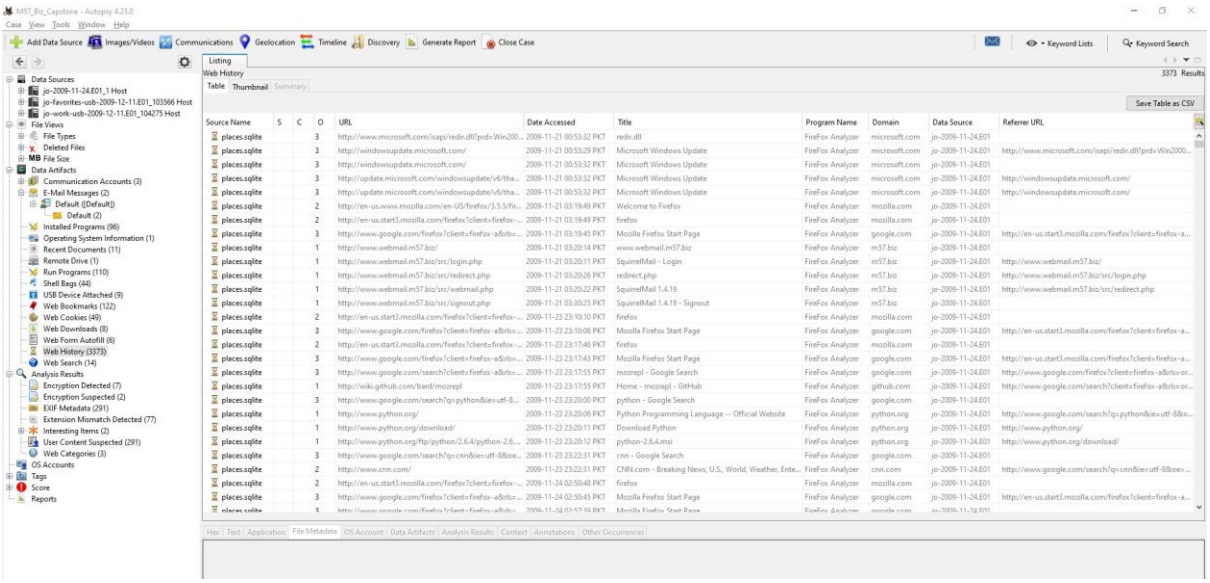
				manipulation utilities.
2009-11-23 21:57:24	User UserAssist Registry	pythonw.exe	Local Terminal File System	First runtime execution record logged for background python interpreter tool.
2009-11-23 23:17:55	places.sqlite Database	wiki.github.com/bard/mozrepl	github.com	User accesses github repository to download browser hijacking configuration addon.

2009-11-23 23:20:12	places.sqlite Database	python-2.6.4.msi	python.org	Jo downloads installer package package to finalize script runtime tools locally.

2009-11-24 03:02:28	Internet Proxy Data Log	netacgi/nph-Parser?s1=time	patft.uspto.gov	Automated background script initializes search strings to generate cover traffic patterns.

2009-11-24 03:09:50	Internet Proxy Data Log	netacgi/nph-Parser?s1=cryptography	patft.uspto.gov	Tool shifts targets, running automated queries for encryption keywords via hijacked browser links.
2009-11-25 02:03:30	places.sqlite Database	netacgi/*********	patft.uspto.gov	The automation code attempts to parse file headers from urls.txt, pushing asterisk strings to
				web servers.

2009-11-25 02:07:22	places.sqlite Database	netacgi/docCrawler...	patft.uspto.gov	Crawler reads header line text strings directly, pushing text data into search engine bars.
2009-11-25 02:17:41	Volume Downloader Manifest	MNspreadsheetTagged.csv	usms.nist.gov	The data collection crawler extracts target row 1 to local download paths.
2009-11-25 02:18:18	Volume Downloader Manifest	Security/webfile/.../STU.doc	www.easc.noaa.gov	The tool retrieves target row 2, downloading the restricted encryption document.

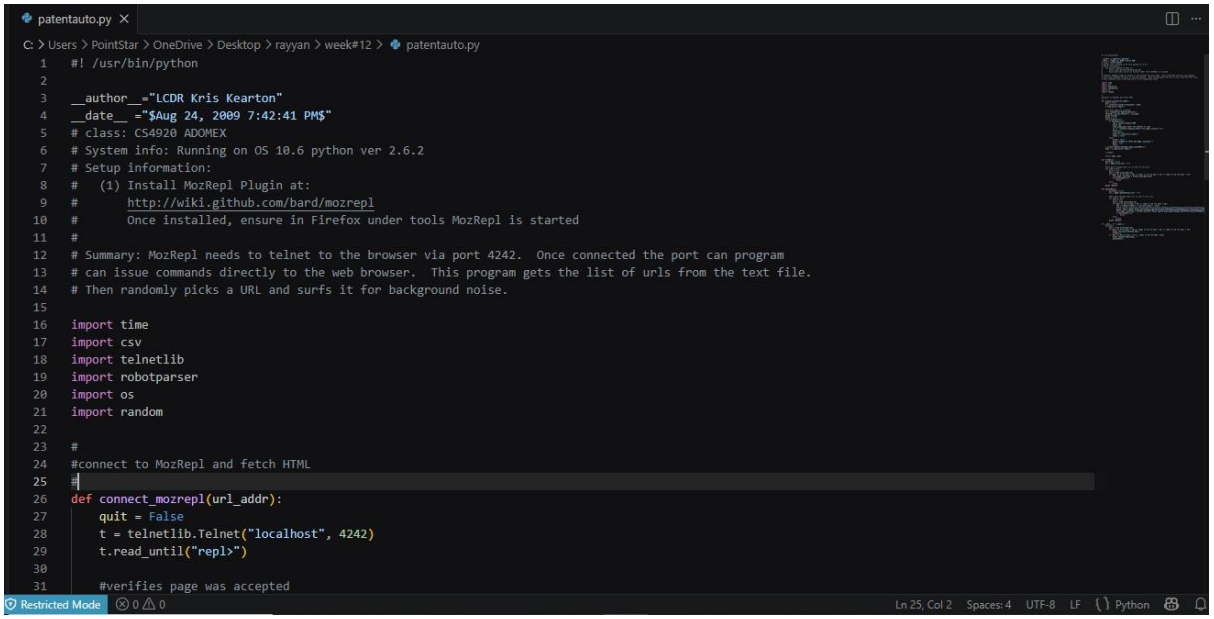


7.2. Inter-Artifact Microtiming Correlation Analysis

A close analysis of the chronology maps out the automated progression of the operation. On November 23rd, between 23:17 and 23:20 PKT, the core script dependencies (python-2.6.4.msi and MozRepl) were manually gathered via the browser interface.

By the morning of November 25th, the execution history logs the automation engine running through its parameters. The parser anomalies logged at 02:03 and 02:07 PKT—where the script read text headers and formatting asterisks from the input file and pushed them directly into browser requests—confirm that an automated script process was controlling the browser window, rather than manual human typing.

This automated routine successfully retrieved the NIST dataset spreadsheet and the NOAA STU-III cryptographic security overview at 02:17 and 02:18 PKT, routing them into the local staging folders.



8. Final Forensic Case Conclusion

8.1. Summary of Proved Intent & Attribution

The combined volatile, non-volatile, registry, and script-layer digital forensic audit confirms an intentional insider data collection event carried out by the subject profile, Jo.

The parsed system metrics expose a pre-planned configuration sequence. On November 23rd, the user set up the required automation dependencies by downloading the MozRepl browser control extension and installing the Python script environment. Following setup, the user deployed an automated collection tool (patentauto.py) guided by targeted index scripts (patentterms.txt, urls.txt).

The synchronized master timeline logs confirm that the automated toolset ran in structured loops on the morning of November 25th. The engine's syntax limitations caused it to pass text file headers directly into browser streams before successfully executing direct download commands. These actions pulled the NIST spreadsheet dataset and the unclassified but controlled Department of Commerce STU-III cryptographic security overview directly into Jo's private folders. In addition, the storage of data hiding utilities (diit-1.5.jar) directly alongside matching runtime records shows a coordinated approach to collecting and concealing restricted assets. The collected forensic evidence establishes clear, automated insider activity.

8.2. Operational Sign-off & Verification Checkpoint Log

- **Registry Hive Configuration Audit: PASSED**
- **Host Run Tracker Deconstruction: COMPLETED**
- **Evidence Staging Payload Extraction: VERIFIED**
- **Case Investigation Status: CLOSED / MULTI-LAYER EVIDENCE WRAPPED FOR EXTRACTION AUDIT**